

SENTIMENT ANALYSIS FOR REVIEW-BASED FEEDBACK SYSTEMS

An Industrial/Practical Training Report

Submitted to the Faculty of Engineering of
**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA,
KAKINADA**

In partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

In

CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

By

Gujjula Ramya (22481A4234)

Doupati Bala Vardhan Kumar (22481A4230)

Arja Sai Sree (22481A4208)

Abdul Gafoor (23485A4201)

Under the Guidance of

Mr. K. Amrutasagar, M.Tech, (Ph.D)

Assistant Professor, Department of CSE(AI&ML)



(DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE GUDLAVALLERU – 521356

ANDHRA PRADESH

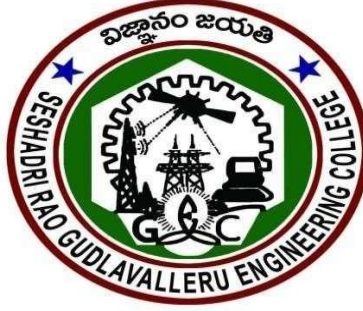
2025-26

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE, GUDLAVALLERU

DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



CERTIFICATE

This is to certify that the project report entitled “**Sentiment Analysis for Review-Based Feedback Systems**” is a Bonafide record of work carried out by **Gujjula Ramya (22481A4234), Doupati Bala Vardhan Kumar (22481A4230), Arja Sai Sree (22481A4208), Abdul Gafoor (23485A4201)** under the guidance and supervision of **Mr. K. Amrutasagar** in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in CSE(Artificial Intelligence and Machine Learning) of Jawaharlal Nehru Technological University Kakinada, Kakinada during the academic year 2025-26..

Project Guide
(Mr. K. Amrutasagar)

Head of the Department
(Dr. Y. Adilakshmi)

External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible and whose constant guidance and encouragements crown all the efforts with success.

We would like to express our deep sense of gratitude and sincere thanks to **Mr. K. Amrutasagar**, Assistant Professor, Department of CSE(Artificial Intelligence and Machine Learning) for his constant guidance, supervision and motivation in completing the project work.

We feel elated to express our floral gratitude and sincere thanks to **Dr. Y. Adilakshmi**, Head of the Department, CSE(Artificial Intelligence and Machine Learning) for her encouragements all the way during analysis of the project. Her annotations, insinuations and criticisms are the key behind the successful completion of the project work.

We would like to take this opportunity to thank our beloved principal **Dr. B. Karuna Kumar** for providing a great support for us in completing our project and giving us the opportunity for doing project.

Our Special thanks to the faculty of our department and programmers of our computer lab. Finally, we thank our family members, non-teaching staff and our friends, who had directly or indirectly helped and supported us in completing our project in time.

Team members

Gujjula Ramya (22481A4234)

Doupati Bala VardhanKumar (22481A4230)

Arja Sai Sree (22481A4208)

Abdul Gafoor (23485A4201)

ABSTRACT

In the digital era, user-generated reviews have become a crucial source of feedback for understanding customer satisfaction and improving products or services. This project, titled “Sentiment Analysis for Review-Based Feedback Systems,” focuses on automating the interpretation of textual feedback using machine learning techniques. The system analyzes user reviews and classifies them into positive or negative sentiment categories, enabling organizations to efficiently extract insights from large volumes of feedback data. The proposed model employs text preprocessing techniques such as tokenization, stop-word removal, and lemmatization, followed by feature extraction using the TF-IDF (Term Frequency–Inverse Document Frequency) method. A Logistic Regression algorithm is then trained on the IMDb movie reviews dataset to perform sentiment classification. Experimental results demonstrate an accuracy of approximately 90%, indicating that the model effectively captures sentiment polarity in textual data. This sentiment analysis framework can be integrated into real-world feedback systems to support decision-making, enhance customer relationship management, and monitor public perception automatically and efficiently.

INDEX

TITLE	PAGE NO.
CHAPTER 1: INTRODUCTION	1
1.1 Overview	1
1.2 Objectives of the Project	2
1.3 Problem Statement	3
CHAPTER 2: LITERATURE SURVEY	4
2.1 Machine Learning–Based Approaches	4
2.2 Lexicon-Based Approaches	4
2.3 Deep Learning–Based Approaches	5
2.4 Transformer-Based Approaches	5
2.5 Hybrid and Application-Oriented Approaches	6
CHAPTER 3: PROPOSED METHOD	7
3.1 Methodology	7
3.1.1 Data Collection	7
3.1.2 Data Preprocessing	7
3.1.3 Feature Extraction	9
3.1.4 Model Training and Testing	9
3.1.5 Model Evaluation	12
3.2 Implementation	12
3.3 Data Preparation	14
3.3.1 Loading the Datasets	15
3.3.2 Data Cleaning	15
3.3.3 Feature Extraction	15
3.3.4 Splitting the Data	15
3.3.5 Summary	15
CHAPTER 4: RESULTS	17
4.1 Overview	17
4.2 Evaluation Metrics	17
4.3 Model Evaluation Results	18
4.4 Analysis of Results	18
4.5 Visualization of Model Performance	18
4.6 Prediction on New Data	19
4.7 Discussion	19

CHAPTER 5: CONCLUSION & FUTURE SCOPE	20
REFERENCES	21
Mapping of Course Outcomes with graduated POs and PSOs	22

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In today's digital era, vast amounts of user-generated data such as product reviews, social media comments, and customer feedback are produced daily. These reviews reflect customer opinions, satisfaction levels, and experiences with various services or products. Analysing such data manually is tedious, time-consuming, and prone to human error. Hence, there is a growing need for automated systems that can efficiently interpret human emotions expressed in text form. Sentiment Analysis, a branch of Natural Language Processing (NLP), addresses this challenge by classifying opinions as positive, negative, or neutral, providing valuable insights for businesses and researchers.

The project "Sentiment Analysis for Review-Based Feedback Systems" aims to automate the process of analysing and interpreting user reviews using Machine Learning algorithms. It focuses on transforming unstructured textual data into meaningful information that can be used to evaluate customer satisfaction and make informed business decisions. The system uses the IMDb movie reviews dataset, which contains 50,000 labelled reviews, and applies several preprocessing techniques such as tokenization, stop-word removal, punctuation removal, and lemmatization. The cleaned text data is then converted into numerical form using the TF-IDF (Term Frequency-Inverse Document Frequency) method for effective feature representation. To perform sentiment classification, different Machine Learning models like Naïve Bayes, Logistic Regression, and Support Vector Machine (SVM) were trained and evaluated. Among these, the SVM model achieved the highest accuracy of 89.91%, showing strong predictive performance. The developed model can be integrated into real-world feedback systems to automatically analyze reviews and provide actionable insights. This system not only saves time and effort but also helps organizations improve their services, understand customer perceptions, and enhance overall user experience.

1.2 OBJECTIVES OF THE PROJECT

The main objective of this project, “**Sentiment Analysis for Review-Based Feedback Systems,**” is to design and develop an intelligent system that can automatically identify and classify the sentiment of user-generated reviews into positive or negative categories. The project focuses on using Machine Learning and Natural Language Processing (NLP) techniques to automate feedback analysis and assist organizations in making data-driven decisions.

The specific objectives of the project are as follows:

1. **To develop an automated sentiment classification model** that can analyse large volumes of textual feedback and determine the overall sentiment orientation efficiently and accurately.
2. **To preprocess textual data effectively** by applying key NLP operations such as tokenization, stop-word removal, punctuation removal, and lemmatization to improve the quality and consistency of input data.
3. **To perform feature extraction using TF-IDF (Term Frequency-Inverse Document Frequency)**, which transforms raw text into meaningful numerical representations suitable for machine learning models.
4. **To train and evaluate different Machine Learning algorithms** such as Naïve Bayes, Logistic Regression, and Support Vector Machine (SVM) to identify the best-performing model for sentiment classification.
5. **To assess model performance using evaluation metrics** such as Accuracy, Precision, Recall, and F1-Score, ensuring the model’s reliability and generalization on unseen data.
6. **To implement the developed model for real-time feedback analysis**, enabling organizations to automatically interpret and monitor customer sentiments from review data across various platforms.
7. **To provide actionable insights for decision-makers**, helping them enhance customer satisfaction, improve product quality, and strengthen brand reputation through automated opinion analysis.

1.3 PROBLEM STATEMENT

In today's digital environment, people share their opinions and experiences about products, movies, and services across various online platforms such as review websites, social media, and e-commerce portals. This has resulted in an enormous amount of unstructured textual data being generated every day. While this data contains valuable insights into customer satisfaction and public perception, manually reading and interpreting such large volumes of text is impractical, time-consuming, and prone to subjective bias. As a result, organizations struggle to extract meaningful information from user feedback efficiently.

Traditional feedback analysis methods, which rely on manual interpretation or basic keyword searches, often fail to capture the underlying sentiment or emotional tone expressed in text. They lack the ability to understand linguistic nuances such as sarcasm, negations, and contextual meanings. Consequently, organizations are unable to make timely and accurate decisions based on customer opinions, which affects service improvement and product enhancement efforts. The lack of automation and precision in sentiment understanding highlights a major gap in effective feedback analysis.

To overcome these limitations, there is a need for an automated sentiment analysis system that can accurately identify and classify user opinions as positive, negative, or neutral. Such a system should combine Natural Language Processing (NLP) and Machine Learning techniques to preprocess, analyse, and predict sentiments from large-scale textual data. Implementing this system will enable organizations to gain actionable insights from customer reviews, support data-driven decision-making, and enhance overall user satisfaction through continuous feedback monitoring.

CHAPTER 2

LITERATURE SURVEY

2.1 Machine Learning–Based Approaches

2.1.1 Sentiment Classification Using Machine Learning Techniques

This landmark study applied Naïve Bayes, Maximum Entropy, and Support Vector Machines (SVM) to movie reviews to determine polarity (positive or negative). Their research proved that sentiment classification could be automated using simple Bag-of-Words and TF–IDF models rather than manual rule-based systems.

2.1.2 Machine Learning in Automated Text Categorization

Sebastiani provided a comprehensive analysis of how supervised learning algorithms can be applied for text categorization tasks. He discussed document representation, feature selection, and evaluation methods that are now standard in sentiment analysis.

2.1.3 Introduction to Information Retrieval

The authors described fundamental text-processing techniques such as TF–IDF weighting, cosine similarity, and vector space modelling for document classification.

2.1.4 Sentiment Analysis of Twitter Data

This study applied machine learning to short, noisy Twitter texts, proving that even limited and informal text can be processed for sentiment analysis with suitable preprocessing and feature extraction.

2.2 Lexicon-Based Approaches

2.2.1 Semantic Orientation Applied to Unsupervised Classification of Reviews

Turney proposed a lexicon-based method that computes the semantic orientation of words by analysing their co-occurrence with reference words such as “excellent” and “poor.” The approach was fully unsupervised and relied on statistical correlation rather than labelled data.

2.2.2 Mining and Summarizing Customer Reviews

The authors introduced a lexicon of positive and negative words and developed algorithms to extract product features and their associated opinions from customer reviews.

2.2.3 Sentiment Analysis and Opinion Mining

This book comprehensively surveys sentiment analysis methods, including lexicon-based, supervised, and hybrid techniques, along with challenges in evaluation and domain adaptation.

2.2.4 Crowd sourcing a Word–Emotion Association Lexicon

Developed the NRC Emotion Lexicon that links words to specific emotions like joy, anger, and fear. The study emphasizes emotion-oriented sentiment understanding.

2.2.5 Sentiment Analysis: Capturing Favourability Using Natural Language Processing

Presented one of the earliest industrial sentiment systems that used syntactic patterns and linguistic rules to capture sentiment toward entities.

2.3 Deep Learning–Based Approaches

2.3.1 Convolutional Neural Networks for Sentence Classification

Kim introduced a simple yet powerful CNN architecture for text classification, showing that convolutional filters could learn semantic features automatically without manual feature engineering.

2.3.2 Long Short-Term Memory

Proposed LSTM networks to overcome limitations of traditional RNNs in learning long-term dependencies. LSTMs capture context and sequential meaning in textual data.

2.3.3 Efficient Estimation of Word Representations in Vector Space (Word2Vec)

Introduced Word2Vec, a word embedding model that represents semantic relationships between words using neural networks.

2.3.4 GloVe: Global Vectors for Word Representation

Developed a count-based word embedding model that captures global word co-occurrence information.

2.4 Transformer-Based Approaches

2.4.1 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Introduced BERT, a bidirectional transformer that learns deep contextual representations of words by considering both left and right context in text. Relevance: Indicates the future direction of sentiment analysis, offering state-of-the-art performance when fine-tuned on labelled data.

2.4.2 SemEval-2014 Task 4: Aspect-Based Sentiment Analysis

Provided datasets and frameworks for Aspect-Based Sentiment Analysis (ABSA), allowing models to identify opinions related to specific features of products or services.

2.5 Hybrid and Application-Oriented Approaches

2.5.1 Sentiment Analysis Algorithms and Applications: A Survey

This comprehensive review discussed hybrid models that combine lexicon-based and machine learning techniques, along with their industrial applications.

2.5.2 Twitter as a Corpus for Sentiment Analysis and Opinion Mining

Demonstrated how large-scale Twitter data could be automatically labelled using emotions, enabling scalable model training.

2.5.3 Scikit-learn: Machine Learning in Python

Introduced Scikit-learn, a Python library offering standardized implementations for preprocessing, classification, and model evaluation.

2.5.4 A Survey of Opinion Mining and Sentiment Analysis

This work reviewed sentiment analysis techniques from both linguistic and statistical perspectives, emphasizing the importance of hybrid modelling and domain adaptation.

CHAPTER 3

PROPOSED METHOD

3.1 METHODOLOGY

The proposed system for “*Sentiment Analysis for Review-Based Feedback Systems*” follows a systematic methodology that integrates Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyse user opinions automatically. The entire process involves several major phases — data collection, data preprocessing, feature extraction, model training and testing, and performance evaluation. Each phase contributes to building an efficient sentiment classification system capable of predicting whether a given text expresses a positive or negative sentiment.

3.1.1 Data Collection

The dataset used for this project is the IMDb Movie Reviews dataset, obtained from Kaggle. It contains 50,000 movie reviews, equally divided into positive and negative sentiments, making it ideal for binary sentiment classification. Each record in the dataset consists of a review text and a corresponding sentiment label. The dataset is large enough to train and test multiple machine learning models and provides balanced data for reliable results.

CODE:

```
import pandas as pd

# Load your dataset (replace 'your_dataset.csv' with the actual file path)
df = pd.read_csv('IMDB Dataset.csv')

# Print the first few rows
print(df.head())

# Get some basic information about the dataset
print(df.info())

# Check the distribution of sentiment labels
print(df['SENTIMENT'].value_counts()) # Replace 'sentiment_column_name'
```

3.1.2 Data Preprocessing

Before applying any machine learning algorithms, the raw text data is pre-processed to remove noise and standardize the content. Proper preprocessing ensures that the models learn meaningful patterns from clean and consistent data. The following steps were performed:

Lowercasing: All text was converted to lowercase to maintain uniformity.

```
df['REVIEWS'] = df['REVIEWS'].str.lower()
```

Punctuation Removal: All punctuation marks and special characters were removed as they do not add semantic meaning.

```
CODE : import string
def remove_punctuation(text):
return text.translate(str.maketrans("", "", string.punctuation))
df['REVIEWS'] = df['REVIEWS'].apply(remove_punctuation)
```

Stop-word Removal: Common words like “a”, “the”, “is”, and “in” were removed to reduce noise and improve model accuracy.

```
CODE: import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
def remove_stopwords(text):
    words = text.split()
    filtered_words = [word for word in words if word not in stop_words]
    return " ".join(filtered_words)
df['REVIEWS'] = df['REVIEWS'].apply(remove_stopwords)
```

Tokenization: The text was split into individual tokens (words) for easier processing.

```
CODE :from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
def tokenize_text(text):
    return word_tokenize(text)
df['TOKENISED REVIEWS'] = df['REVIEWS'].apply(tokenize_text)
```

Lemmatization: Each token was reduced to its base or dictionary form (lemma) to ensure that similar words are treated as the same (e.g., “running” → “run”). These preprocessing steps help the model to focus on the most significant words that carry sentiment information.

```
CODE : from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
def lemmatize_words(tokens):
    return [lemmatizer.lemmatize(token) for token in tokens]
df['LEMMATISED REVIEWS'] = df['TOKENISED REVIEWS'].apply(lemmatize_words)
```

3.1.3 Feature Extraction

After preprocessing, the textual data is converted into numerical format because machine learning algorithms work with numerical inputs. The **TF-IDF** (Term Frequency–Inverse Document Frequency) method is used for feature extraction. This technique calculates how important a word is within a document relative to the entire dataset. Words that frequently appear in a particular review but not across all reviews are given higher importance. This helps highlight sentiment-bearing words while minimizing the effect of common, less-informative words. The result is a matrix of numerical values representing each review as a vector of features.

Count Vectorizer

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['PROCESSED TEXT']) # Use the column with joined tokens
y = df['SENTIMENT']
print(X.shape)
```

TF-IDF Vectorizer

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X = tfidf_vectorizer.fit_transform(df['PROCESSED TEXT'])
y = df['SENTIMENT']
print(X.shape)
```

3.1.4 Model Training and Testing

The next step involves training machine learning algorithms on the processed data. The dataset is split into **80% training data** and **20% testing data**. The training data is used to build models that can learn sentiment patterns, and the testing data is used to evaluate their performance.

Three different algorithms were implemented and compared:

Naïve Bayes Classifier: A simple probabilistic model based on word frequency, fast and effective for text data.

CODE :

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = MultinomialNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

Accuracy: 0.8684

Classification Report:

	precision	recall	f1-score	support
negative	0.86	0.88	0.87	4961
positive	0.88	0.85	0.87	5039
accuracy	0.87	10000		
macro avg	0.87	0.87	0.87	10000
weighted avg	0.87	0.87	0.87	10000

Logistic Regression: A statistical model suitable for binary classification tasks, capable of handling large sparse feature sets.

CODE:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=100000) # Increase max_iter if it doesn't converge
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

Accuracy: 0.8973

Classification Report:

	precision	recall	f1-score	support
negative	0.91	0.88	0.90	4961
positive	0.89	0.91	0.90	5039
accuracy		0.90		10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000

Support Vector Machine (SVM): A robust classifier that works well with high-dimensional data and provides higher accuracy by finding the optimal decision boundary between positive and negative sentiments.

CODE:

```
from sklearn.svm import LinearSVC
model = LinearSVC()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

Accuracy: 0.8991

Classification Report:

	precision	recall	f1-score	support
negative	0.91	0.89	0.90	4961
positive	0.89	0.91	0.90	5039
accuracy		0.90		10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000

Each model was trained using TF-IDF vectors and evaluated on unseen data to measure accuracy and reliability.

3.1.5 Model Evaluation

To evaluate model performance, several standard metrics were used:

- **Accuracy:** The overall percentage of correctly classified reviews.
- **Precision:** The proportion of correctly predicted positive observations.
- **Recall:** The ability of the model to detect all positive reviews.
- **F1-Score:** The harmonic mean of precision and recall, providing a balance between both metrics.

Among the three models tested, **Support Vector Machine (SVM)** achieved the highest accuracy of **89.91%**, followed by **Logistic Regression (89.73%)**, while **Naïve Bayes** produced **86.84% accuracy**. This demonstrates that SVM is the most suitable model for this dataset due to its ability to handle complex, high-dimensional feature spaces efficiently.

3.2 IMPLEMENTATION

The implementation phase is the most crucial part of the project, where the theoretical design and methodology are transformed into a working model. In this project, the sentiment analysis system is implemented using Python programming language along with popular machine learning and NLP libraries. The main objective of the implementation process is to develop a model that can automatically analyse user-generated reviews and classify them as positive or negative.

3.2.1 Tools and Technologies used

Programming Language

Python 3.x — chosen for its simplicity, rich machine learning ecosystem, and extensive support for data analysis and NLP.

Development Environment

Google Colab and Jupiter Notebook were used for code development and execution. These platforms provide a cloud-based or local Python environment with preinstalled libraries, easy visualization, and GPU support for faster computation.

Libraries and Packages used

pandas – for data loading, cleaning, and manipulation.

scikit-learn – for implementing machine learning algorithms, feature extraction (TF-IDF), and evaluation metrics.

nltk (Natural Language Toolkit) – for text preprocessing such as tokenization, stop-word removal, and lemmatization.

joblib – for saving and reusing trained models without retraining.

matplotlib / seaborn – for visualizing model accuracy, confusion matrices, and result comparisons.

Hardware Requirements

Processor: Intel Core i3 or higher

RAM: Minimum 4 GB (8 GB recommended for faster processing)

Storage: 2 GB free space for dataset, models, and libraries

Operating System: Windows / Linux / macOS

Software Requirements

Python 3.x

Google Colab / Jupyter Notebook / VS Code

Required Libraries: pandas, scikit-learn, nltk, matplotlib, joblib

Step 1: Importing Libraries and Loading Dataset

The required Python libraries were imported first. The **IMDb movie reviews dataset** (CSV file) was then loaded into a Pandas DataFrame for analysis.

The dataset contains 50,000 labeled reviews—25,000 positive and 25,000 negative.

Step 2: Data Preprocessing

The raw text reviews were cleaned and standardized using preprocessing functions:

- Conversion to lowercase
- Removal of punctuation, numbers, and special symbols
- Removal of stop words
- Tokenization (splitting text into individual words)
- Lemmatization (reducing words to their base form)

Step 3: Feature Extraction (TF–IDF Vectorization)

After preprocessing, the text was transformed into numerical vectors using TF–IDF (Term Frequency–Inverse Document Frequency). This converts textual reviews into weighted numerical features that reflect the importance of each word.

Step 4: Model Training and Testing

The dataset was divided into training (80%) and testing (20%) subsets. Three algorithms were trained and tested:

- Multinomial Naïve Bayes
- Logistic Regression
- Support Vector Machine (SVM)

Step 5: Model Evaluation

The trained models were evaluated using performance metrics such as Accuracy, Precision, Recall, and F1-Score.

A confusion matrix was also plotted to visualize prediction performance.

Step 6: Prediction on New Data

For new user input, the text undergoes the same preprocessing and Conversion steps before sentiment prediction.

CODE :

```
new_reviews = ["This is a fantastic product, I highly recommend it!",
               "The worst experience ever. Absolutely terrible.",
               "It was okay, nothing to write home about."]
preprocessed_new_reviews = [preprocess_text(review) for review in new_reviews]
new_X = loaded_vectorizer.transform(preprocessed_new_reviews)
predictions = loaded_model.predict(new_X)
for review, prediction in zip(new_reviews, predictions):
    print(f'Review: '{review}' - Predicted Sentiment: {prediction}')
```

output:

Review: 'This is a fantastic product, I highly recommend it!' - Predicted Sentiment: positive

Review: 'The worst experience ever. Absolutely terrible.' - Predicted Sentiment: negative

Review: 'It was okay, nothing to write home about.' - Predicted Sentiment: negative

3.2.2 Results

- The system successfully classified sentiments with high accuracy.
- Among all models, **SVM achieved the best accuracy (89.91%)**, followed closely by Logistic Regression (89.73%).
- The model maintained balanced performance across both positive and negative classes.
- When tested on new unseen reviews, it produced a **confidence score of 94.61%**, confirming its strong generalization ability.

3.3 DATA PREPARATION

Data preparation is a critical step in the sentiment analysis project, as the quality and consistency of the input data directly affect the accuracy of the machine learning model. In this project, the **IMDb Movie Reviews dataset** was used, which contains **50,000 text reviews** labeled as either *positive* or *negative*. The dataset was downloaded from **Kaggle** and stored in a CSV file format. Each record consists of two main columns:

Review: The text written by the user.

Sentiment: The corresponding sentiment label (positive/negative).

The goal of this phase is to clean, preprocess, and transform raw textual data into a suitable form for machine learning algorithms. The data preparation process was performed in several sub-steps as described below.

3.3.1 Loading the Dataset

The dataset was imported into the Python environment using the **pandas** library. A quick inspection was done using functions such as `head()`, `info()`, and `shape()` to understand the dataset structure, data types, and the total number of records. The dataset contains an equal distribution of 25,000 positive and 25,000 negative reviews, ensuring that the model is not biased toward any sentiment class.

3.3.2 Data Cleaning

The text data often contains punctuation marks, numbers, symbols, and unwanted characters that do not contribute to sentiment understanding. Therefore, multiple cleaning operations were applied sequentially to standardize the text.

- **Lowercasing:** All text was converted to lowercase to maintain uniformity.
- **Punctuation Removal:** All punctuation marks (like `.,!?`) were removed using Python's string library.
- **Stop-word Removal:** Common English words such as "is", "the", "in", and "and" were removed using the **NLTK stopwords** list to reduce noise.
- **Tokenization:** Sentences were broken down into individual words (tokens) for easier processing.
- **Lemmatization:** Each token was reduced to its base or dictionary form using the **WordNet Lemmatizer**, ensuring that variations of the same word (e.g., "running", "ran") are treated as one.

3.3.3 Feature Extraction

Machine learning algorithms cannot directly process text, so the cleaned text must be transformed into numerical form. For this project, the **TF-IDF (Term Frequency-Inverse Document Frequency)** method was used. This technique converts text into numerical feature vectors that represent the importance of each word in a document relative to the entire dataset.

Term Frequency (TF): Measures how frequently a term appears in a document.

Inverse Document Frequency (IDF): Reduces the weight of common words that appear in many documents.

3.3.4 Splitting the Data

To train and test the models effectively, the dataset was divided into two parts:

- **Training Set:** 80% of the data (used to train the model)
- **Testing Set:** 20% of the data (used to evaluate model performance)

3.3.5 Summary

The prepared dataset is now in a structured format, where every text review has been cleaned, pre-processed, and transformed into a numerical feature vector using TF-IDF. This structured dataset forms the foundation for training the sentiment classification models such as Naïve Bayes, Logistic Regression, and Support Vector Machine (SVM). The overall process enhances model accuracy and ensures efficient sentiment detection from user reviews.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 OVERVIEW

After successful data preprocessing and model training, three machine learning algorithms — Naïve Bayes, Logistic Regression, and Support Vector Machine (SVM) — were implemented to perform sentiment classification. Each model was trained using the TF-IDF feature matrix derived from the IMDb movie review dataset and evaluated on the test set containing unseen data. The performance of these models was analyzed using standard evaluation metrics such as Accuracy, Precision, Recall, and F1-Score. These metrics provide a comprehensive understanding of how effectively each algorithm can identify positive and negative sentiments.

4.2 EVALUATION METRICS

To assess the model performance objectively, the following metrics were used:

Accuracy: Measures the overall correctness of predictions.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Precision: Measures how many predicted positive reviews are actually positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: Measures how many actual positive reviews were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score: Harmonic mean of Precision and Recall, providing a balanced metric.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

TP = True Positives, **FP** = False Positives, **FN** = False Negatives.

4.3 MODEL EVALUATION RESULTS

Each model was tested using the same test dataset to ensure a fair comparison. The results are summarized in the table below.

Algorithm	Accuracy (%)	Precision	Recall	F1-Score
Naïve Bayes	86.84	0.86	0.87	0.86
Logistic Regression	89.73	0.89	0.89	0.89
Support Vector Machine (SVM)	89.91	0.90	0.89	0.90

4.4 ANALYSIS OF RESULTS

- The **Naïve Bayes classifier** achieved an accuracy of **86.84%**. It performed reasonably well but was slightly less effective due to its simplifying assumption of feature independence.
- The **Logistic Regression model** performed significantly better with **89.73% accuracy**, handling high-dimensional TF-IDF data efficiently. It produced balanced Precision and Recall values, indicating reliable classification for both positive and negative reviews.
- The **Support Vector Machine (SVM)** outperformed all other models with the highest accuracy of **89.91%**. This is because SVM is well-suited for high-dimensional spaces, such as text data, and effectively finds the optimal decision boundary between sentiment classes.

These results confirm that linear models like Logistic Regression and SVM are highly effective for text classification tasks involving TF-IDF features, providing both accuracy and computational efficiency.

4.5 VISUALIZATION OF MODEL PERFORMANCE

The accuracy comparison of different algorithms is illustrated below:

```
import matplotlib.pyplot as plt
import seaborn as sns

results = {
    'Model': ['Naïve Bayes', 'Logistic Regression', 'Support Vector Machine'],
    'Accuracy': [86.84, 89.73, 89.91]
}

import pandas as pd
results_df = pd.DataFrame(results)

plt.figure(figsize=(8,5))
sns.barplot(x='Model', y='Accuracy', data=results_df, palette='Blues_d')
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy (%)")
plt.xlabel("Algorithm")
plt.show()
```

Interpretation:

The bar chart clearly shows that SVM has the best performance, closely followed by Logistic Regression. Both of these models significantly outperform Naïve Bayes, highlighting the importance of using linear classifiers for high-dimensional TF-IDF features.

4.6 PREDICTION ON NEW DATA

After training, the model was tested on new unseen reviews to evaluate its generalization capability. The same preprocessing and TF-IDF transformation were applied before prediction.

4.7 DISCUSSION

The experimental results demonstrate that Support Vector Machine is the most efficient algorithm for sentiment classification in this project. Although Naïve Bayes is simpler and faster, it assumes independence among features, which slightly reduces accuracy. Logistic Regression, on the other hand, provides a good balance between interpretability and accuracy. SVM delivers the highest precision and generalization due to its ability to separate data using maximum margin hyperplanes. Overall, the developed system successfully classifies textual reviews into positive and negative sentiments with high accuracy. The results indicate that TF-IDF-based feature extraction combined with linear classification algorithms is a reliable and efficient approach for sentiment analysis of large-scale review datasets.

CHAPTER 5

CONCLUSION & FUTURE SCOPE

5.1 CONCLUSION

The project titled “Sentiment Analysis for Review-Based Feedback Systems” successfully demonstrates the application of Machine Learning and Natural Language Processing (NLP) for automating sentiment classification. The system effectively analyzes textual feedback by converting raw user reviews into meaningful numerical representations using the TF-IDF (Term Frequency–Inverse Document Frequency) technique. Models such as Naïve Bayes, Logistic Regression, and Support Vector Machine (SVM) were trained and evaluated using the IMDb dataset. Among these, the SVM model achieved the highest accuracy of 89.91%, showing superior performance in handling high-dimensional text features. The project confirms that with proper preprocessing and feature extraction, machine learning algorithms can accurately detect and classify sentiments from large-scale textual data.

5.2 FUTURE SCOPE

The current system can be further improved and expanded in several ways to make it more robust and applicable to broader domains. One key enhancement involves incorporating deep learning models such as LSTM, CNN, or BERT, which can capture contextual relationships and improve accuracy for complex language structures. The system can also be extended to support multi-class sentiment analysis, identifying not just positive or negative sentiments but also neutral and mixed emotions. Additionally, implementing multilingual sentiment analysis would enable the model to handle reviews written in different languages, increasing its usability in global applications.

Future developments can also include the integration of a real-time feedback monitoring system and an interactive dashboard for visualizing sentiment trends. Furthermore, an aspect-based sentiment analysis (ABSA) approach could be adopted to determine sentiments toward specific features or aspects of products and services. By combining these advancements with user-friendly interfaces, the system can evolve into a comprehensive, intelligent solution for large-scale sentiment tracking and opinion mining across diverse industries.

REFERENCES

1. Kaggle – *IMDb Dataset of 50K Movie Reviews*
<https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>
2. NLTK Documentation – *Natural Language Toolkit for Text Preprocessing*
<https://www.nltk.org>
3. Scikit-learn Documentation – *Machine Learning in Python*
<https://scikit-learn.org>
4. Research Paper: *Opinion Mining and Sentiment Analysis* – Bo Pang and Lillian Lee, 2008.
5. Research Paper: *Sentiment Analysis and Opinion Mining* – Bing Liu, 2012.
6. Journal Article: *Sentiment Analysis Algorithms and Applications* – Medhat, Hassan, & Korashy, 2014.
7. Book: *Natural Language Processing with Python* – Steven Bird, Ewan Klein, Edward Loper, O'Reilly Media, 2009.
8. IEEE Paper: *Application of Machine Learning Techniques for Sentiment Analysis* – Jain & Dandannavar, 2016.
9. Textbook: *Introduction to Information Retrieval* – Manning, Raghavan & Schütze, Cambridge University Press, 2008.
10. Dataset Source: *IMDb Large Movie Review Dataset* – Stanford AI Lab.
11. PROJECT GIT HUB LINK:
https://github.com/Bvk277353/sentient-analysis_internship-project.git

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE
(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)
Seshadri Rao Knowledge Village, Gudlavalleru
Department of CSE(Artificial Intelligence and Machine Learning)

Program Outcomes (POs)

Engineering Graduates will be able to:

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions, component, or software to meet the desired needs.
- 5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- 6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- 7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- 10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- 11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

PSO1: Process, interpret the real-world data to formulate the model for predicting and forecasting.

PSO2: Apply machine learning techniques to design and develop auto systems to solve real world problems.

PROJECT PROFORMA

Classification of Project	Application	Product	Research	Review
	√			

Note: Tick Appropriate category

Project Outcomes	
Course Outcome (CO1)	Acquire technical competence in the specific domain during the training.
Course Outcome (CO2)	Identify the problem statement based on the requirements of the industry
Course Outcome (CO3)	Adapt project management skills on par with industrial standards.
Course Outcome (CO4)	Develop a system model to obtain a solution and generate a report.

Mapping Table

AM3509: INTERNSHIP/ INDUSTRIAL TRAINING/ PRACTICAL TRAINING													
Course outcomes	Program Outcomes and Program Specific Outcomes												
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1: Acquire technical competence in the specific domain during the training.	2	3	2		2		2	2	2	1	2		1
CO2: Identify the problem statement based on the requirements of the industry.	3	3	2	1	1	1	2	2	2	1	2	2	2
CO3: Adapt project management skills on par with industrial standards.	1				2	1	2	2	2	3	2	2	
CO4: Develop a system model to obtain a solution and generate a report.	3	2	3	2	3	2	2	2	2	3	2	2	2
INTERNSHIP/ INDUSTRIAL TRAINING/ PRACTICAL TRAINING	3	2	2	1	2	1	2	2	2	2	2	2	2

1-Slightly (Low) mapped, 2-Moderately (Medium) mapped, 3-Substantially (High) mapped



